

SONIC GAME



Introduction :

Pour ce projet, nous avons choisi de revisiter un grand classique du jeu vidéo : Sonic. Dans notre version, le joueur incarne Sonic, qui progresse dans une version 2D un peu comme à la RPG. Son objectif est de collecter un maximum d'anneaux pour ne pas risquer sa jauge de vie lorsqu'il affronte Eggman, lui fallant minimum 5 anneaux avant de le débloquent. Il peut aussi faire la sieste, manger des chili dog ou chercher des boost (speed token)/checkpoint tout comme il aime faire dans le vrai jeu. Notre but principal était surtout de bien comprendre comment utiliser ANTLR et faire le lien entre notre fichier .py et g4. Au début on pensait à un langage pour des recettes mais on a vu que c'était très vu et revu sur internet, optant alors pour un jeu qu'on peut modéliser comme on veut et de façon plus originale.

Description :SonicRPG.g4 :

Le lexer : son rôle est de transformer une chaîne de caractères entrée par le joueur en une séquence de tokens reconnaissables par le parser. Les règles du lexer sont écrites en majuscule et triées selon leur « type » :

- *Mots-clé du jeu* : commandes principales que l'utilisateur peut écrire pour contrôler les actions de Sonic.
- *Direction* : commandes qui permettent à l'utilisateur de choisir les directions de déplacement de Sonic.
- *Opérateurs de comparaison / opérateurs arithmétiques* : utile pour le programme, notamment lors de conditions logiques, mais l'utilisateur ne les utilise pas directement dans la console.
- *Ponctuations* : permettent de structurer correctement les instructions, notamment pour écrire des coordonnées.
- *Types de données* : permet de différencier et reconnaître plusieurs types de données.
- *Espaces et commentaires* : gère les commentaires se trouvant dans le programme.

Le parser : son rôle est de recevoir des tokens et vérifier s'ils respectent la structure grammaticale attendue. Notre parser contient 15 types d'instructions différentes :

- *MoveStatement* : déplace Sonic dans une direction donnée
- *CollectStatement* : ramasse les anneaux se trouvant sur le chemin
- *UseStatement* : permet d'utiliser les « speed »
- *FightStatement* : déclenche le combat contre le boss
- *SayStatement* : affiche des messages à l'utilisateur au fil de la partie
- *IfStatement* : exécute les instructions seulement si une certaine condition est vraie
- *LoopStatement* : exécute un bloc d'instructions un certain nombre de fois
- *ExitStatement* : termine la partie
- *LoadStatement* : charge une sauvegarde
- *InventoryStatement* : affiche l'inventaire de Sonic

- *WaitStatement* : permet de faire une pause dans la partie
- *StatusStatement* : permet de voir l'état du joueur
- *ExpressionStatement* : permet d'affecter une valeur à une variable

SonicInterpreter.py :

Les méthodes du visiteur correspondent directement aux règles de grammaire définies dans SonicRPG.g4, avec chaque méthode visit traitant une commande ou une construction spécifique du jeu. Lorsqu'un joueur entre une commande, le parser généré par ANTLR crée un arbre d'analyse, que le visiteur parcourt ensuite pour exécuter la logique de jeu correspondante.

La méthode visitMoveStatement traite les commandes de mouvement en mettant à jour les coordonnées de la position de Sonic dans l'état du jeu, tout en vérifiant les limites de la carte. L'implémentation des mouvements prend en charge à la fois les directions NORTH, SOUTH, EAST, WEST et UP, DOWN, RIGHT, LEFT.

Le système de collection, géré par visitCollectStatement, illustre l'interaction de l'interpréteur avec le monde du jeu. Cette méthode vérifie la position actuelle à la recherche d'objets collectables, met à jour l'inventaire du joueur ou ses statistiques (comme le nombre d'anneaux), et retire les objets collectés de la carte du jeu. Une logique de collection spéciale existe pour différents types d'objets :

- Les anneaux augmentent le compteur du joueur,
- Les jetons de vitesse sont ajoutés à l'inventaire,
- Les coffres au trésor peuvent contenir des peluches spécifiques à des personnages, qui servent d'objets de collection.

La méthode visitUseStatement gère la commande "USE" dans le jeu Sonic RPG. Elle permet au joueur d'utiliser différents objets qu'il a collectés. Par exemple, utiliser un "SPEED_TOKEN" augmente le multiplicateur de vitesse de Sonic, le faisant déplacer plus vite. La méthode vérifie si le joueur possède l'objet dans son inventaire avant de l'utiliser et fournit un retour pour les tentatives réussies ou échouées. Elle permet également de désactiver les boosts de vitesse et de sauvegarder des points de contrôle à la position actuelle.

La méthode visitFightStatement gère la commande "FIGHT BOSS". Cette méthode vérifie si Sonic a collecté suffisamment d'anneaux pour débloquent le combat contre le boss et s'il se trouve au bon endroit (la zone du boss à la position 5,5). Si les conditions sont remplies, le combat commence et la puissance de Sonic est calculée en fonction de ses anneaux, de sa vitesse et de sa santé, tandis que le boss a une puissance aléatoire comprise entre 60 et 100. Si Sonic gagne, le jeu se termine par une victoire ; s'il perd, il subit des dégâts compris entre 20 et 40 et peut obtenir un "Game Over" si sa santé tombe à zéro.

Dans le code, nous avons également créé une méthode visitWaitStatement qui gère la commande "WAIT". Lorsqu'elle est utilisée, Sonic récupère lentement de la santé (5 HP) jusqu'à

son maximum de 100 HP. Nous avons ajouté cette commande pour que le joueur ait plus de chances de gagner contre Eggman.

La méthode `visitStatusStatement` affiche les statistiques actuelles de Sonic dans le jeu, notamment sa position, les anneaux collectés, le multiplicateur de vitesse, sa santé et l'état du boss, tout en affichant les objets de l'inventaire. La méthode `visitHelpStatement` liste toutes les commandes disponibles, comme se déplacer, collecter des objets, combattre le boss et utiliser des boucles/conditions. Cette méthode guide les joueurs sur la façon d'interagir avec le jeu.

L'une des méthodes importantes est la méthode `_execute_command`, qui est le processeur de commandes du jeu – elle prend des entrées texte brutes comme "MOVE DOWN" et les transforme en actions du jeu. Lorsqu'un joueur tape une commande, cette méthode active les outils linguistiques d'ANTLR : d'abord, le lexer décompose le texte en tokens, puis le parser vérifie les règles de grammaire, et enfin le visiteur exécute la logique réelle du jeu. Elle compile des commandes simples comme "COLLECT RING" en opérations complexes du jeu. Si l'entrée ne correspond pas aux règles linguistiques du jeu, elle détecte l'erreur et suggère de consulter le menu HELP.

La classe `GameState` sauvegarde les données du joueur, tel que sa position, sa santé, les objets qu'il a ramassés, la carte du monde, etc... Par exemple la méthode `_generate_world()` place des objets sur la carte (rings, speed tokens, checkpoints, des chili dogs, et des trésors) à des endroits précis.

La fonction `display_map()` affiche notre carte du jeu en ASCII où chaque symbole représente quelque chose (🦔 pour Sonic, 🍍 pour les rings, 📦 pour les trésors etc...). Cela nous permet de nous repérer. Il y a aussi une fonction `get_current_position_objects()` qui regarde ce qu'il y a à l'endroit où Sonic se trouve et nous propose directement de les COLLECT.

La classe `SonicRPGGame` est la base du jeu. Elle commence l'aventure avec `start_game()` c'est elle qui lit ce que le joueur écrit et va faire appel à l'ANTLR pour exécuter les commandes demandées. C'est elle qui montre les messages tel que bienvenu, niveau de vie trop faible ou victoire etc... Ainsi que des petits éléments pour rendre le jeu plus fluide (checkpoint/sauvegarde) même si elles ne sont pas très utiles dans ce cas-là au vu de la longueur du jeu, les checkpoints sont aussi surtout pour la référence au vrai jeu.

Il y a des fonctions comme `visitCondition`, `visitExpression` et `visitAtom` qui permettent d'évaluer des conditions ou des calculs (-> séries vues en classe). Par exemple, on peut écrire des commandes comme `IF rings > 3 { MOVE RIGHT }` ou répéter une action avec `LOOP 3 { MOVE DOWN }`. Ce n'est pas obligatoire pour jouer et pas très intuitif, mais ça permet de faire des scripts plus complexes, comme dans les exercices qu'on a vus en cours.

Exemple d'aventure SonicRPG:

Le jeu vient de commencer et vous souhaitez que Sonic parle. La fonction `SAY` nous permet de le faire. Pour cela, suivre la structure suivante :

SAY "message"

SAY "Bienvenue dans l'aventure de Sonic!"

SAY "Objectif: Collecter 5 rings pour débloquent le boss!"

On affiche le statut (position, nombre de rings/speed tokens collectés, boss débloquent/vaincu ou non...), ici initial puisque nous venons de commencer :

STATUS

On commence par l'exploration de notre environnement

SAY "Phase 1 : Exploration de la map"

MAP

Pour se déplacer, On utilise MOVE, suivi de la direction et le nombre de pas (entre 1 et 9).

MOVE RIGHT/LEFT 3

MOVE UP/DOWN 5

Pour collecter un ring, il suffit d'aller sur la case où se trouve le ring et utiliser COLLECT :

COLLECT RING (ou RINGS)

Pour collecter un speed token, on remplace RING par SPEED_TOKEN. Pour l'utiliser, exécuter la fonction USE, puis on se déplace avec la fonction MOVE. Et pour revenir à la vitesse normale, il suffit de *turn off* le token.

COLLECT SPEED_TOKEN

USE SPEED_TOKEN

MOVE NORTH 4

USE SPEED OFF

Les checkpoints permettent de sauvegarder notre progression. Lorsqu'on arrive sur un checkpoint, il suffit d'exécuter USE.

USE CHECKPOINT

SAY "Checkpoint activé pour sauvegarder ma progression"

Si vous voulez automatiser des actions que vous répétez un certain nombre de fois à la suite, il suffit d'utiliser LOOP, qui prend comme paramètres le nombre de loops et les actions

LOOP 3 {

MOVE DOWN 2

COLLECT RINGS

MOVE RIGHT 1

COLLECT RINGS

MOVE UP 1

}

ENDLOOP

Maintenant qu'on a collecté les 5 rings nécessaires, on peut aller combattre Eggman. Tant qu'on n'a pas collecté 5 rings (minimum), le combat ne peut pas commencer. On vérifie aussi notre statut, que tout est bon et on est prêt.

STATUS

Dès qu'on est prêt, on se dirige vers Eggman, situé au centre de la map (5,5). Il faut être sur la même case pour pouvoir se battre.

SAY "Je me dirige vers Eggman"

Maintenant qu'on se trouve à la position (5,5), il est temps d'affronter Eggman. Pour cela, on utilise la fonction *FIGHT BOSS*.

SAY "Il est temps d'affronter le boss final !"

FIGHT BOSS

Si le combat échoue, on conseille, ou recharge notre énergie grâce à *WAIT*, qui prend comme paramètre le temps (en secondes) que vous voulez passer à vous reposer.

WAIT 5

Au cas où vous perdez une – ou plusieurs – vie(s) due à des défaites répétées, les chili-dogs vont nous en redonner. Il suffit – comme pour les rings et speed tokens – utiliser la fonction *COLLECT* (et se trouver sur la case du chili dog).

COLLECT CHILI_DOG

Des trésors peuvent être aussi collectés avec *COLLECT*.

COLLECT TRESOR

Maintenant qu'on a fini, on peut vérifier notre statut. Pour sortir du jeu, il suffit d'utiliser *EXIT*.

STATUS

EXIT

Amélioration/Conclusion :

Comme tout projet, le nôtre présente des pistes d'amélioration. Nous aurions par exemple pu enrichir l'expérience de jeu en ajoutant plusieurs niveaux, avec une difficulté croissante. Cela aurait permis de varier le nombre d'anneaux à collecter et d'augmenter progressivement la puissance d'Eggman. Peut-être laisser le joueur choisir quel personnage incarner au début, et une fois l'aventure complétée avec tous les personnages un niveau secret se débloquerait (comme dans Sonic Adventure). L'introduction d'obstacles supplémentaires, à l'image du jeu original, aurait également rendu le défi plus intéressant et fidèle à l'univers de Sonic. Beaucoup de concept peuvent encore être exploré d'un point de vue jeu. L'amélioration du code serait de l'alléger, éviter les *Visit* inutile, essayer de reprendre les classes déjà existantes et juste y ajouter des variables plutôt qu'à chaque fois en faire une nouvelle. Au début on a perdu beaucoup de temps à comprendre comment faire correctement le *visit* et s'adapter à cette manière d'écrire en python, qui diffère de ce qu'on a vu d'habitude étant donné qu'on dépend de l'ANTLR.